Introduction to Web Technologies

SWE230 :
WEB APPLICATION PROGRAMMING

Spring 2026

Instructors

- **Sarah Nabil**
  Sara.abdullah@miuegypt.edu.eg
- **Nada Ayman**
  nada.ayman@miuegypt.edu.eg
- **Nada AbdelFattah**
  nada.abdelfattah@miuegypt.edu.eg

In this Lecture

- **Introduction**
- **JavaScript Language**
- Introduction
- Inserting JS in HTML
- JS Basics
- Variables
- Arrays
- Loops
- Functions
- Arrow Functions
- Objects

In this Lecture

- Web Application
- Client-Side JS: Document Object Model
- Events
- Form Validation
- Timing Events
- Ajax

JAVASCRIPT

Introduction to JavaScript

JavaScript History

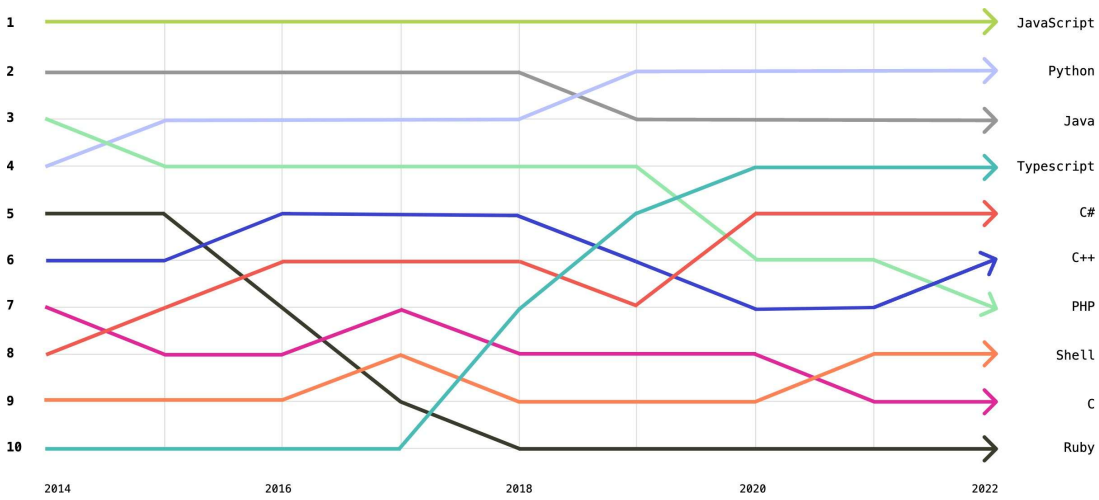
- JavaScript was developed in 1995 by Brendan Eich at **Netscape**.
- It was originally going to be called LiveScript.
  But because Sun's Java language was the new big thing at the time, Netscape and Sun agreed to rename LiveScript to JavaScript.
- Java and JavaScript are completely different.
- Microsoft made their own version Jscript into IE 3.
- First standardized edition of JavaScript was officially called ECMAScript, and it was first published in 1997.
- **ECMAScript** Editions is now supported by all popular browsers.

Terminology

- **JavaScript**: The commonly used name for implementations of the ECMAScript standard. This term isn't tied to a particular version
- **ECMAScript**: A language standardized by ECMA International
- **ECMAScript 5 (ES5)**: The 5th edition of ECMAScript, standardized in 2009.
- **ECMAScript 6 (ES6)/ ECMAScript 2015 (ES2015)**: The 6th edition of ECMAScript, standardized in 2015.
  - **ECMAScript 2020**(ES2020)

https://en.wikipedia.org/wiki/ECMAScript

What can JavaScript be used for



https://octoverse.github.com/2022/top-programming-languages#:~:text=Top%20languages%20used%20in%202022,place%20year%2Dover%2Dyear

JavaScript is a popular and versatile programming language that can be used for

- web development (frontend & backend).
- data analysis.
- game development.

The JavaScript Language

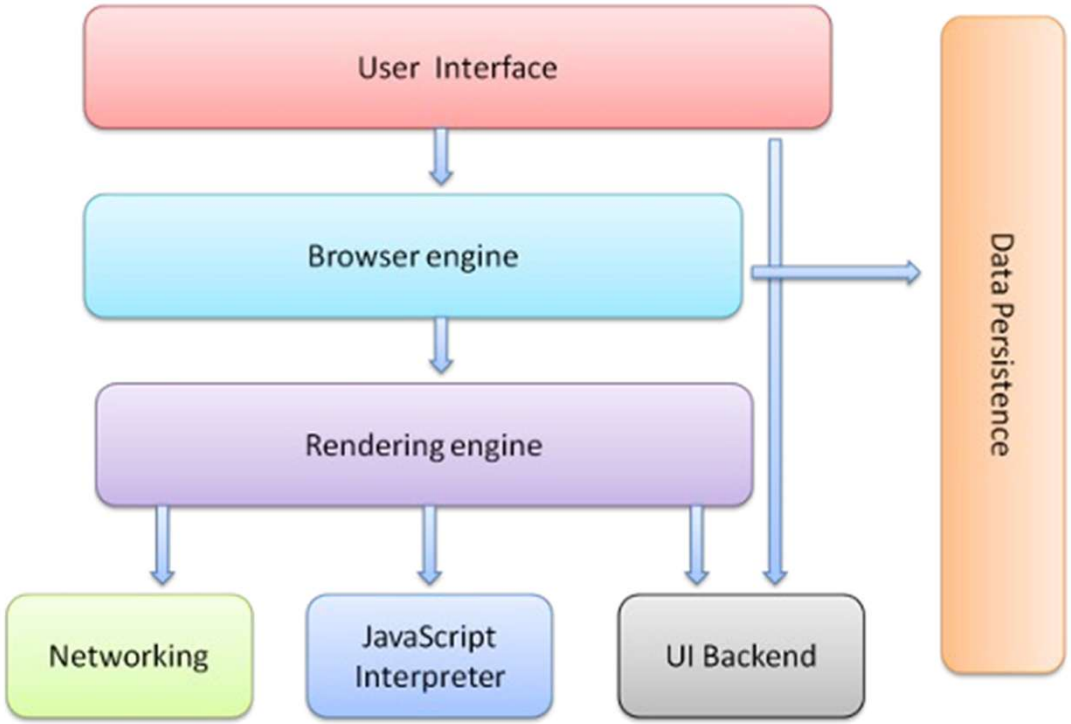
- The JavaScript language has become increasingly popular and is one of the most prominent programming language on the Web.

- Client-side JavaScript is a free, scripting language that lets you add **interactivity** to Hypertext Markup Language (HTML) pages.
- It adds behavior to otherwise a static web content.

- Client-side means that the JavaScript language runs in the browser and is not used on the server side. Google Maps, for example, uses JavaScript to allow users to interact with a map by moving it around, zooming in and out, and so on.

JavaScript Engine

A **JavaScript engine** is like a virtual machine that interprets and executes JavaScript. Although there are several uses for a JavaScript engine, it is most commonly used in web browsers.



What is Chrome V8

- Chrome V8 is a JavaScript engine, which means that it executes JavaScript code. Originally, JavaScript was written to be executed by web browsers. Chrome V8, or just V8, can execute JavaScript code either within or outside of a browser, which makes server-side scripting possible.

- Chrome V8 is fast and powerful. V8 translates JavaScript code directly into machine code so that computers can actually understand it, then it executes the translated code.

- V8 optimizes JavaScript execution as well.

JavaScript is a Scripting Language

- JavaScript (JS) is a lightweight, interpreted, or just-in-time (JIT) compiled programming language.
- While it is most well-known as the scripting language for Web pages, many non-browser environments also use it, such as Node.js, Apache CouchDB and Adobe Acrobat.
- JavaScript is the world's most popular programming language [1].
- JavaScript is the programming language of the Web.

1) https://www.statista.com/statistics/793628/worldwide-developer-survey-most-used-languages/

JavaScript as a client-side language

- JavaScript provides user interactivity with a web page after the page loads, developers commonly use it for some of the following functions:

Including JavaScript in HTML

JavaScript including in HTML

- To include a JavaScript code in an HTML file, you must place the code inside script tags and include the text/javascript type attribute

1- EventHandler - by using the onevent attribute like onclick inside HTML elements
2- Internal - by using a <script> element in body or head tags

```
<script type="text/javascript">
   var x = 5;
    ……
 </script>
```

3- External - by using a <script> element with src attribute to link to an external file

```
<script type="text/javascript"
        src="js/javascript.js"> </script>
```

Why use External JavaScript files

External JavaScript files are the most common way to include JavaScript code for a number of practical reasons:

- Search engines can crawl and index your web site faster if there is less code within your HTML page.

- Keeping your JavaScript code separate from your HTML is cleaner and ultimately easier to manage.

- Because you can include multiple JavaScript files in your HTML code, you can separate the JavaScript files into different folder structures on your web server, similar to images, which is an easier way to manage your code. Clean, organized code is always key to easily managing your website.

JavaScript Basics

JavaScript Syntax

- All JavaScript statements **should** end with a semicolon.
- **The language is case sensitive.**
- document.write() function is used to display dynamic content through JavaScript.
- For fast input and output, prompt and console.log is used

index.html

```
10   document.write("Hello JS");
```

JS COMMENT

- The JavaScript comments are meaningful way to deliver message
- It is used to add information about the code, warnings or suggestions
- **The JavaScript comment is ignored by the JavaScript engine**

index.html

```
// It is single line comment
document.write("Hello JS");
/* It is multi line comment.
It will not be displayed */
```

JS VARIABLES

- A JavaScript variable is simply a name of storage location
- Name must start with a letter (a to z or A to Z), underscore(_), or dollar($) sign
- Subsequent characters may be letters, digits, underscores, or dollar signs.
- **JavaScript variables are case sensitive, for example x and X are different variables.**

p1.html

```
var x = 10;
var y = 20;
var z=x+y;
document.write(z);
```

Java Script DATA TYPES

Data Type	Description
String	represents sequence of characters e.g. "hello"
Number	represents numeric values e.g. 100
Boolean	represents boolean value either false or true
Undefined	represents undefined value
Null	represents null i.e. no value at all

other DATA TYPEs

The non-primitive data types are as follows

Data Type	Description
Object	represents instance through which we can access members
Array	represents group of similar values
RegExp	represents regular expression

null and undefined

- Both *null* and *undefined* can be regarded as a special value that indicates “no value”.
- The null is a language keyword that is used to indicate the expected lack of value. On the other hand, undefined is a predefined global variable that has a deeper meaning. It is used to indicate an error- like, unexpected lack of value.

- When your function has no return value, it returns undefined. If you declare a variable and don’t initialize it, it returns the undefined value. If you query a non-existent array element or object property, an undefined value is returned.

JavaScript Variable declaration and usage

```
var x; //x is undefined here
x = 5;
var y = 6;
var z = x + y;
var pi = 3.14;

//Strings are text, written within double or single quotes:
var str = "CS";
str = str + " - " + "web development"
```

JavaScript Data Types

```
var length = 16;                                    // Number
var lastName = "Johnson";                           // String
var cars = ["Saab", "Volvo", "BMW"];                // Array
var person = {firstName:"John", lastName:"Doe"};     // Object
var flag = true;                                      // boolean
```

```
typeof “Ahmed"                  // Returns string
typeof 3.14                     // Returns number
```

Arrays

- Arrays are like variables, but different in that they can hold multiple values and expressions under one name.
- Storing multiple values in one variable is what makes arrays so powerful.

- While arrays can hold any data type in the JavaScript language, including other arrays, it is most common to store similar data in any one array and to assign it a name that is related to the item it contains

Arrays

- While it may be possible to store all these items in one array, it isn't logical and can cause problems later in the script, such as identifying what data is in the array.

```
<script type="text/javascript">
    let colors = new Array("orange", "blue", "red", "brown");
    let shapes = new Array("circle", "square", "triangle");
</script>
```

```
console.log("Red: "+ colors[2]);
console.log("Brown: "+ colors[3]);
```

Accessing array elements

```
var colors = new Array();
colors[0] = "orange";
colors[1] = "blue";
colors[2] = "red";
colors[3] = "brown";
console.log("Blue: "+ colors[1]);

// Updating blue to purple
colors[1] = "purple";
console.log("Purple: "+ colors[1]);
```

Array properties and methods

```
//Add a new element to the end of the array
.push
//Prints the length of array
.length
//Removes the element at the end of the array
.pop
//Reverses the order of elements in the array
.reverse
//Sorts the elements of an array
.sort
//combines two or more arrays and returns a new string.
.concat
//filter and extract the element of an array that satisfying the
provided condition.
.filter
//returns the first element of the given array that satisfies the
provided function condition.
```

Exercise: what is the output

```
Let students = ['Zyad', 'Eman', 'Maha'];

console.log("Length of original students' array: " + students.length);
students.push('Salem');

console.log("Length of students array after adding an element: "
            + students.length);

students.pop();//Remove element at the end of the array
console.log(students);

students.reverse();
console.log(students);

students.sort();
console.log(students);
```

String properties and Methods

- Length
- Slice()
- Split()
- indexOf()
- lastIndexOf()
- trim()

Variables Scope

- Variables in JavaScript can be declared with the **var** , **let or const** keyword.

- The difference is scoping. var is scoped to the nearest function block and let is scoped to the nearest *enclosing* block, which can be smaller than a function block. Both are global if outside any block.

```
<script type="text/javascript">
    var cost = 120, profit = 77;
    let x = 5;
    str = “test”;
</script>
```

Variables Scope

```
const x = 6; //similar to C or C++'s `const`
function fun() {
   if (x > 1) {
      let m = "Using Let";
      var v = "Using Var";
      y = "no var or let used";
   }

   console.log("inside fun x = " + x);
   console.log("inside fun y = " + window.y);
   console.log("inside fun v = " + v);
   console.log("inside fun m = " + m);//Uncaught ReferenceError: m is not defined
}
fun();
console.log("outside fun x = " + x);
console.log("outside fun y = " + window.y);
console.log("outside fun v = " + v); //Uncaught ReferenceError: v is not defined
```

Use const, let, or var ?

- let, behave in terms of scope as you would expect a java or C++ variable. **It is recommended to use it whenever possible.**

- var has scope in functions meaning the variables are valid within the function and not outside the function.

- Use const if you need to declare a constant (value will be initialized once and do not change afterwards). Its scope is similar to let to the nearest *enclosing* block.
- Using const, let, or var outside a function creates a global variable
- Var allows redeclaration while Let doesn’t allow redeclaration but allows nested block shadowing

Scope Rules

- In JavaScript, scope is the set of variables, objects, and functions you have access to.
- Variables declared within a JavaScript function, become LOCAL to the function (must use var).
- A variable declared outside a function, becomes GLOBAL
- If you assign a value to a variable that has not been declared, it will automatically become a GLOBAL variable

- The lifetime of a JavaScript variable starts when it is declared.
  - Local variables are deleted when the function is completed.
  - Global variables are deleted when you close the page.

JAVASCRIPT OPERATORS

- JavaScript operators are symbols that are used to perform operations on operands

  - Arithmetic Operators
  - Comparison (Relational) Operators Bitwise Operators
  - Logical Operators
  - Assignment Operators Special Operators

JAVASCRIPT ARITHMETIC OPERATORS

Arithmetic operators are used to perform arithmetic operations on the operands. The following operators are known as JavaScript arithmetic operators.

Operator	Description	Example
+	Addition	10+20 = 30
-	Subtraction	20-10 = 10
*	Multiplication	10*20 = 200
/	Division	20/10 = 2
%	Modulus (Remainder)	20%10 = 0
++	Increment	var a=10; a++; Now a = 11
--	Decrement	var a=10; a--; Now a = 9

JAVASCRIPT COMPARISON OPERATORS

The JavaScript comparison operator compares the two operands. The comparison operators are as follows:

Operator	Description	Example
==	Is equal to	10==20 = false
===	Identical (equal and of same type)	10==20 = false
!=	Not equal to	10!=20 = true
!==	Not Identical	20!==20 = false
>	Greater than	20>10 = true
>=	Greater than or equal to	20>=10 = true
<	Less than	20<10 = false
<=	Less than or equal to	20<=10 = false

Equals operator (== vs ===)

- JavaScript has two sets of equality operators: === and !==,
  and their evil twins == and !=
- The identity (===) operator behaves identically to the equality (==) operator except no type conversion is done, and the types must be the same to be considered equal.
- If the two operands are of the same type and have the same value,
  then === produces true and !== produces false.

- The equality (==) operator does the right thing when the operands are of the same value, but if they are of different types, they attempt converting the values types.

JAVASCRIPT BITWISE OPERATORS

The bitwise operators perform bitwise operations on operands. The bitwise operators are as follows:

Operator	Description	Example
&	Bitwise AND	(10==20 & 20==33) = false
|	Bitwise OR	(10==20 | 20==33) = false
^	Bitwise XOR	(10==20 ^ 20==33) = false
~	Bitwise NOT	(~10) = -10
<<	Bitwise Left Shift	(10<<2) = 40
>>	Bitwise Right Shift	(10>>2) = 2
>>>	Bitwise Right Shift with Zero	(10>>>2) = 2

JAVASCRIPT LOGICAL OPERATORS

The following operators are known as JavaScript logical operators

Operator	Description	Example
&&	Logical AND	(10==20 && 20==33) = false
||	Logical OR	(10==20 || 20==33) = false
!	Logical Not	!(10==20) = true

JAVASCRIPT ASSIGNMENT OPERATORS

The following operators are known as JavaScript assignment operators

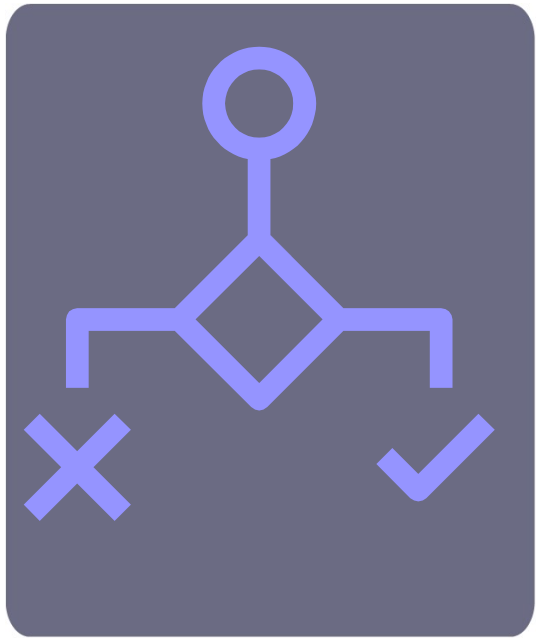
Operator	Description	Example
=	Assign	10+10 = 20
+=	Add and assign	var a=10; a+=20; Now a = 30
-=	Subtract and assign	var a=20; a-=10; Now a = 10
*=	Multiply and assign	var a=10; a*=20; Now a = 200
/=	Divide and assign	var a=10; a/=2; Now a = 5
%=	Modulus and assign	var a=10; a%=2; Now a = 0

Conditional Statements

- **if**: Used to execute a script if a specific condition is true.
- **if...else:** Used to decide which script to execute. The script under the if statement is executed if the condition provided is true, or else the script under the else tag is executed if the condition is false.
- **if...else if...else:** Adds more conditions on top of the if..else statement.
- **switch**: It is an alternative to the if...else if...else statements. However, it does not take in conditions like the if..else statements, but rather uses expressionless values to jump to a particular script.

JAVASCRIPT IF-ELSE

The JavaScript if-else statement is used to execute the code whether condition is true or false.



1.If Statement

2.If else statement

3.if else if statement

JAVASCRIPT IF-ELSE

IF STATEMENT :
It evaluates the content only if expression is true.

```
var age=20;
if(age>18){
document.write("You are adult");
}
```

IF...ELSE STATEMENT:
It evaluates the content whether condition is true of false.

```
var age=20;
if(age>18){
    document.write("You are adult");
}
else{
    document.write("You are child");
}
```

IF...ELSE IF STATEMENT:
It evaluates the content only if expression is true from several expressions.

```
var a=20;
if(a==10){
    document.write("a is equal to 10");
}
else if(a==15){
    document.write("a is equal to 15");
}
else if(a==20){
    document.write("a is equal to 20");
}
else{
    document.write("a is not equal to 10, 15 or 20");
}
```

JAVASCRIPT SWITCH

The JavaScript switch statement is used to execute one code from multiple expressions. It is just like else if statement

index.html

```
switch(grade){
    case 'A':
    result="A Grade";
    break;
    case 'B':
    result="B Grade";
    break;
    case 'C':
    result="C Grade";
    break;
    default:
    result="No Grade";
}
```

JavaScript Loop

- for - loops through a block of code a number of times
- for/in - loops through the properties of an object
- for/of - loops through the values an iterable object

```
for (i=1; i<=5; i++){
    document.write(i + "<br/>")
}
```

```
const person = { name: "Alice", age: 30 };

// Loop over OBJECT keys
for (const key in person) {
  console.log(key); // Output: "name", "age"
  console.log(person[key]); // Output: "Alice", 30
```

```
const colors = ["red", "green", "blue"];

// Loop over ARRAY values
for (const color of colors) {
  console.log(color); // Output: "red", "green", "blue"
}

// Works with strings
const message = "Hi";
for (const char of message) {
  console.log(char); // Output: "H", "i"
}
```

JavaScript Loop

- while - loops through a block of code while a specified condition is true

```
var i=11;
while (i<=15) {
    document.write(i + "<br/>");
    i++;
}
```

- do/while - also loops through a block of code while a specified condition is true

```
var i=21;
do{
    document.write(i + "<br/>");
    i++;
}
while (i<=25);
```

Functions

- A JavaScript function is a block of code designed to perform a particular task. A function is executed when “something” invokes it (calls it).

- Functions are useful for many reasons. They are containers for script that run only by an event or a call to the function. Therefore, functions do not execute when the browser initially loads and executes the script that is included in the web page.

Functions

- Functions also have the ability to accept data through parameters.
- Functions can have one or more parameters, and a function call can have one or more arguments based on the number of function parameters.
- It is common to confuse **parameters** and **arguments**; *parameters* are part of the function definition, and *arguments* are expressions used when calling the function.

Function Call

- A function declaration binds a function to an identifier.
- A function declaration does not ask the code inside the function body to execute.
- The function body runs, or *executes*, only when the function is *called*.

```
① function getGreeting() {
③     console.log("Hello, World!");
  }

② getGreeting();

④ // Code after function call
```

Example

```
let num = 10;

function increase(_num)
{
  _num++;
  console.log("_num is " + _num);
}

console.log("num is " + num);
increase(num);
console.log("num is: "+ num); //call by value
```

Example

```
var num = 10;
var numObject = [10, 15, 20];

function increase(_num, _numObject)
{
  _num++;
  _numObject[0]++;

  console.log("_num is " + _num + ", and _numObject[0] is "
              + _numObject[0]);
}
increase(num, numObject);
console.log("num is " + num + ", and numObject[0] is "
            + numObject[0]);
```

Example

```
var numObject = [10, 15, 20];

function changeArray(_numObject)
{
  _numObject = new Array('turtle', 'chicken', 'kangaroo');
  console.log("_numObject[0] is " + _numObject[0]);
}
changeArray(numObject);
console.log("numObject[0] is " + numObject[0]);
```

```
_numObject[0] is turtle
numObject[0] is 10
```

Arrow Functions (ES6)

- The syntax for an arrow function expression does not require the function keyword and uses a fat arrow **=>** to separate the parameter(s) from the body.

```
// Arrow function with two parameters
const sum = (firstParam, secondParam) => {
        return firstParam + secondParam;
};
console.log(sum(2,5)); // Prints: 7

// Arrow function with no parameters
const printHello = () => {
        console.log('hello');
};
printHello(); // Prints: hello
```

Arrow Functions Return Value by Default:

- If the function has only one statement, and the statement returns a value, you can remove the brackets and the return keyword.

```
hello = () => {
  return "Hello World!";
}
//The RETURN keyword can be omitted here
hello = () => "Hello World!";

//Arrow Function With Parameters
hello = (val) => "Hello " + val;
```

JAVASCRIPT OBJECTS

A javaScript object is an entity having state and behavior (properties and method)

index.html

```
var Person={
    Name:"Rabbil Hasan",
    Age:30,
    City:"Dhaka",
    Country:"Bangladesh"
}
```

JAVASCRIPT ARRAY

JavaScript array is an object that represents a collection of similar type of elements

It has a property called length contains the length of the array, it can be set and delete elements beyond the new length

index.html

```
var city=["Dhaka","Rajshahi","Rangpur"];

for (i=0;i<city.length;i++){

document.write(city[i] + "<br/>");

}
```

JAVASCRIPT ARRAY CONCAT()

The JavaScript array concat() method combines two or more arrays and returns a new string.

index.html

```
var arr1=["A","B","C"];
var arr2=["D","E","F"];
var result=arr1.concat(arr2);
document.writeln(result);
```

JAVASCRIPT ARRAY FROM()

The from() method creates a new array that holds a copy from an array or iterable object.

When applied to a string, each word gets converted to an array element in the new array

index.html

```
var arr=Array.from("RABBIL HASAN");
document.write(arr);
```

JAVASCRIPT ARRAY FILTER()

The JavaScript array filter() Creates a **new array** with only elements that satisfy a condition.

index.html

```
var marks=[50,40,45,37,20];

function check(value)
{
  return value>30;
}

document.writeln(marks.filter(check));
```

JAVASCRIPT ARRAY map()

The JavaScript array **map() - Transform each element**

Creates a **new array** by applying a function to every element.

```
const numbers = [1, 2, 3, 4, 5];

// Double each number
const doubled = numbers.map(num => num * 2);
console.log(doubled); // [2, 4, 6, 8, 10]
```

JAVASCRIPT ARRAY reduce()

The JavaScript array reduce()
Reduces an array to a **single value** (can be number, object, array, etc.).

```
const numbers = [1, 2, 3, 4, 5];

// Sum all numbers
const sum = numbers.reduce((accumulator, current) => accumulator + current, 0);
console.log(sum); // 15
```

JAVASCRIPT ARRAY FIND()

The JavaScript array find() method returns the first element of the given array that satisfies the provided function condition.

index.html

```
var arr=[5,22,19,25,34];
var result=arr.find(x=>x>20);
document.writeln(result)
```

JAVASCRIPT ARRAY FINDINDEX()

The JavaScript array findIndex() method returns the index of first element of the given array that satisfies the provided function condition. It returns -1, if no element satisfies the condition.

index.html

```
var arr=[5,22,19,25,34];
var result=arr.findIndex(x=>x>20);
document.writeln(result
```

JAVASCRIPT ARRAY FOREACH()

The JavaScript array forEach() method is used to invoke the specified function once for each array element.

index.html

```
var arr=[5,22,19,25,34];
arr.forEach(function(fetch) {
  document.writeln(fetch);
});
```

JAVASCRIPT ARRAY INCLUDES()

The JavaScript array includes() method checks whether the given array contains the specified element. It returns true if an array contains the element, otherwise false.

index.html

```
var arr=[5,22,19,25,34];
var result=arr.includes(22);
document.writeln(result);
```

JAVASCRIPT ARRAY INDEXOF()

The JavaScript array indexOf() method is used to search the position of a particular element in a given array. This method is case-sensitive.

index.html

```
var arr=[5,22,19,25,34];
var result= arr.indexOf(19);
document.writeln(result);
```

JAVASCRIPT ARRAY POP()

The JavaScript array pop() method removes the last element from the given array and return that element.

index.html

```
var arr=[5,22,19,25,34];
arr.pop();
document.writeln(arr);
```

JAVASCRIPT ARRAY PUSH()

The JavaScript array push() method adds one or more elements to the end of the given array.

index.html

```
var arr=[5,22,19,25,34];
arr.push(50);
document.writeln(arr);
```

JAVASCRIPT ARRAY REVERSE()

The JavaScript array reverse() method changes the sequence of elements of the given array and returns the reverse sequence.

index.html

```
var arr=[5,22,19,25,34];
array.reverse()
document.writeln(arr);
```

JAVASCRIPT ARRAY SORT()

The JavaScript array sort() method is used to arrange the array elements in some order and modify the original array. By default, sort() method follows the ascending order.

index.html

```
var arr=[5,22,19,25,34];
var result=arr.sort();
document.writeln(result);
```

JAVASCRIPT ARRAY SLICE()

The JavaScript array slice() method extracts the part of the given array and returns it.

This method doesn't change the original array.

arr.slice(startIndex, endIndex)

the optional parameters

- startIndex inclusive
- endIndex is exclusive

index.html

```
var arr=[5,22,19,25,34];
var result=arr.slice(1,3);
document.writeln(result);
```

JAVASCRIPT ARRAY SPLICE()

The JavaScript array splice() method is used to add/remove the elements to/from the existing array. It returns the removed elements from an array.

The splice() method also modifies the original array.
arr.splice(startIndex, deleteCount, item1, item2, ...)

- `startIndex`: Start position
- `deleteCount`: Number of elements to remove
- `items`: Elements to add (optional)

Returns:
Array of removed elements
(empty if none removed).

index.html

```
var arr=["A","B","C","D","E"];
var result=arr.splice(2,0,"B")
document.writeln(arr);
```

JAVASCRIPT STRING METHODS

The JavaScript string is an object that represents a sequence of characters

charAt()
It provides the char value present at the specified index.

concat()
It provides a combination of two or more strings.

indexOf()
It provides the position of a char value present in the given string.

lastIndexOf()
It provides the position of a char value present in the given string by searching a character from the last position.

replace()
It replaces a given string with the specified replacement.

substr()
It is used to fetch the part of the given string on the basis of the specified starting position and length.

JAVASCRIPT STRING METHODS

The JavaScript string is an object that represents a sequence of characters

substring()
It is used to fetch the part of the given string on the basis of the specified index.

slice()
It is used to fetch the part of the given string. It allows us to assign positive as well negative index.

toLowerCase()
It converts the given string into lowercase letter.

toUpperCase()
It converts the given string into uppercase letter.

split()
Divides the string into array of substrings

trim()
It trims the white space from the left and right side of the string.

JAVASCRIPT DATE OBJECT

The JavaScript date object can be used to get year, month and day.

```
const now = new Date();
```

getDate()
It returns the integer value between 1 and 31 that represents the day for the specified date on the basis of local time.

getDay()
It returns the integer value between 0 and 6 that represents the day of the week on the basis of local time.

getFullYears()
It returns the integer value that represents the year on the basis of local time.

getHours()
It returns the integer value between 0 and 23 that represents the hours on the basis of local time.

getMilliseconds()
It returns the integer value between 0 and 999 that represents the milliseconds on the basis of local time.

JAVASCRIPT DATE OBJECT

The JavaScript date object can be used to get year, month and day.

getMinutes()
It returns the integer value between 0 and 59 that represents the minutes on the basis of local time.

getMonth()
It returns the integer value between 0 and 11 that represents the month on the basis of local time.

getSeconds()
It returns the integer value between 0 and 59 that represents the seconds on the basis of local time.

JAVASCRIPT DATE OBJECT

The JavaScript date object can be used to get year, month and day.

getUTCDate()
It returns the integer value between 1 and 31 that represents the day for the specified date on the basis of universal time.

getUTCDay()
It returns the integer value between 0 and 6 that represents the day of the week on the basis of universal time.

getUTCFullYears()
It returns the integer value that represents the year on the basis of universal time.

getUTCHours()
It returns the integer value between 0 and 23 that represents the hours on the basis of universal time.

getUTCMinutes()
It returns the integer value between 0 and 59 that represents the minutes on the basis of universal time.

getUTCMonth()
It returns the integer value between 0 and 11 that represents the month on the basis of universal time.

JAVASCRIPT MATH

The JavaScript math object provides several constants and methods to perform mathematical operation.

abs()
It returns the absolute value of the given number.

ceil()
It returns a smallest integer value, greater than or equal to the given number.

floor()
It returns largest integer value, lower than or equal to the given number.

max()
It returns maximum value of the given numbers.

min()
It returns minimum value of the given numbers.

random()
It returns random number between 0 (inclusive) and 1 (exclusive).

round()
It returns closest integer value of the given number.

JAVASCRIPT NUMBER OBJECT

The JavaScript number object enables you to represent a numeric value. It may be integer or floating-point.

isFinite()
It determines whether the given value is a finite number.

isInteger()
It determines whether the given value is an integer.

parseFloat()
It converts the given string into a floating point number.

parseInt()
It converts the given string into an integer number.

toFixed()
It returns the string that represents a number with exact digits after a decimal point.

toString()
It returns the given number in the form of string.

Creating JavaScript objects

A javaScript object is an entity having state and behavior (properties and method)

```
var person = {
    firstName : “Ahmed",
    lastName  : “Khaled",
    age       : 23,
    sid       : 8976
};

document.getElementById("demo").innerHTML =
person.firstName + " is " + person.age + " years old.";
```

```
typeof person                        // Returns object
```

Adding objects methods

```
var person = {
    firstName : “Ahmed",
    lastName  : “Khaled",
    age       : 23,
    sid       : 8976,
    getFullName  : function() {
       return this.firstName + " " + this.lastName;
    }

};

document.getElementById("demo").innerHTML = person.getFullName();
```

Before the ‘class’ keyword

```
> function Cat(name){
      this.name = name;
  }
<· undefined
> Cat.prototype.meow = function meow(){
      console.log(this.name + ' meow !');
  }
<· ƒ meow(){
      console.log(this.name + ' meow !');
  }
> let cat = new Cat('Hector')
<· undefined
> cat.meow()
  Hector meow !
<· undefined
>
```

ES6 Classes

- ECMAScript 2015, also known as ES6, introduced JavaScript Classes

```
// Declaration
class Rectangle {
  constructor(height, width) {
    this.height = height;
    this.width = width;
  }
 // Getter
  get area() {
    return this.calcArea();
  }
  // Method
  calcArea() {
    return this.height * this.width;
  }
}
```

```
 const r = new Rectangle(5,10);

document.getElementById('out').in
nerHTML = r.area;
```

Document Object Model

Introduction

Presentation
(Style)

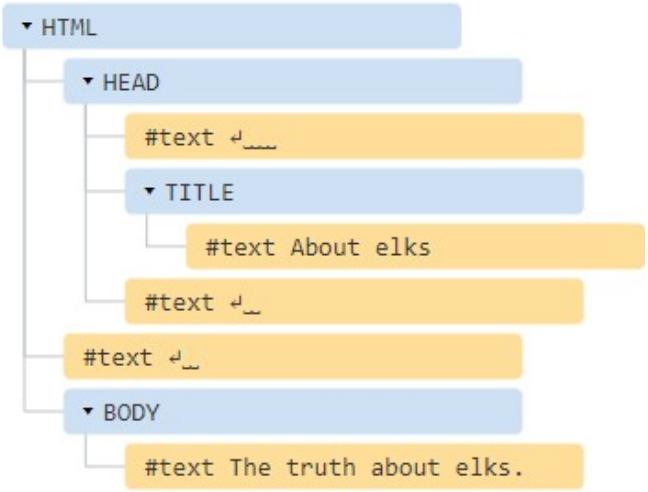
Content
(Structure and semantics)

Behavior

The HTML **DOM** can be accessed with **JavaScript**.
In the **DOM**, all HTML elements are defined as objects.
The programming interface is the properties and methods of each object.

Example of DOM

```
<!DOCTYPE HTML>
<html>
<head>
  <title>About elks</title>
</head>
<body>
  The truth about elks.
</body>
</html>
```



Document Object Model(DOM)

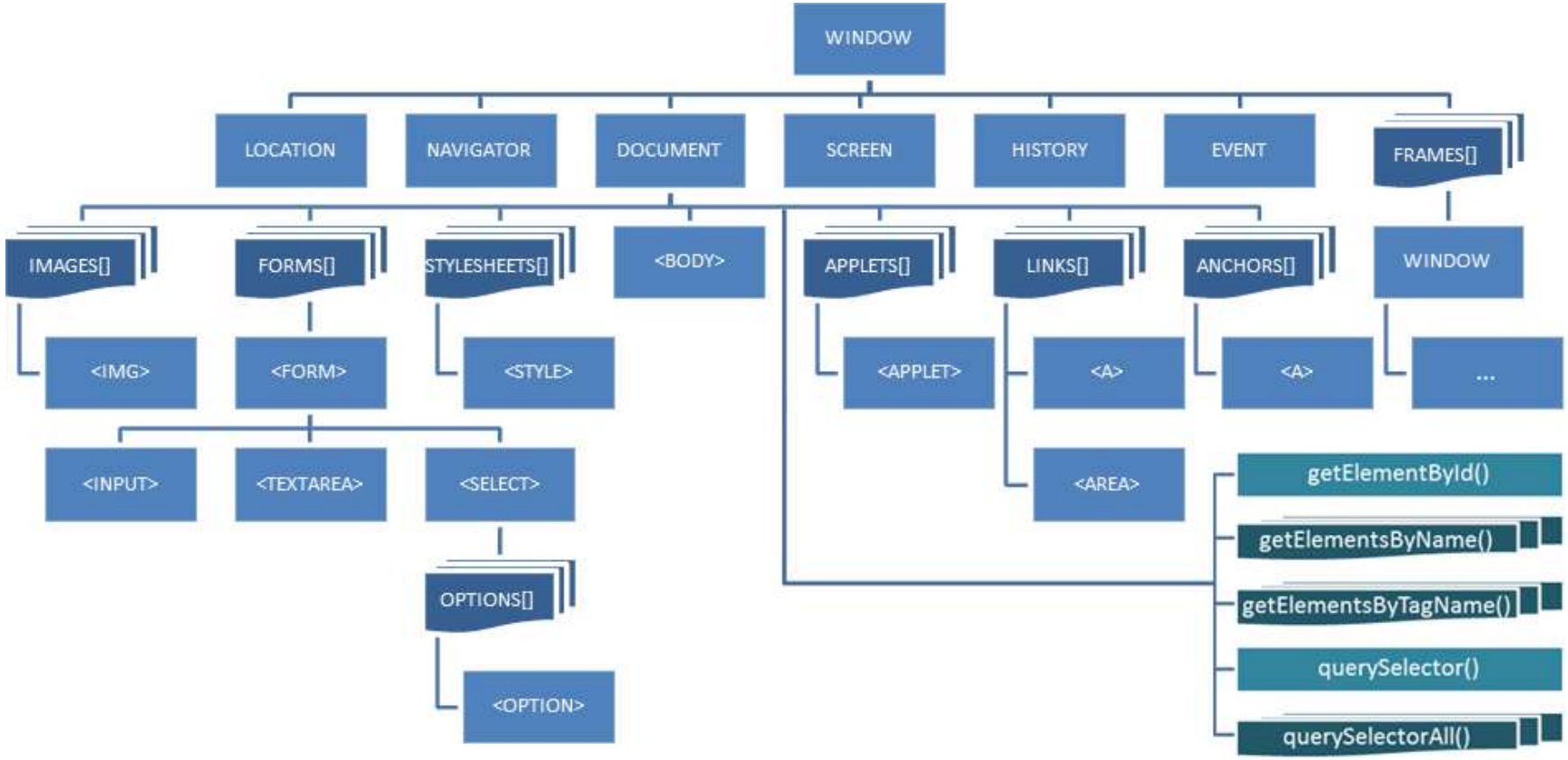
Content
(Structure and semantics)

Presentation
(Style)

Behavior

The HTML **DOM** can be accessed with **JavaScript**.
In the **DOM**, all HTML elements are defined as objects.
The programming interface is the properties and methods of each object.

Document Object Model



What is the HTML DOM?

- The HTML DOM is a standard object model and programming interface for HTML. It defines:
  - The HTML elements as objects
  - The properties of all HTML elements
  - The methods to access all HTML elements
  - The events for all HTML elements

JavaScript and HTML DOM

- The HTML DOM is constructed as a tree of Objects

- With the **D**ocument **O**bject **M**odel, JavaScript gets all the power it needs to create dynamic HTML such as:
  - Change all the HTML elements in the page
  - Change all the HTML attributes in the page
  - Change all the CSS styles in the page
  - Remove existing HTML elements and attributes
  - Add new HTML elements and attributes

document.write

- document.write() can be used to write directly to the HTML output stream.

```
<script>

document.write(Date());
</script>
```

JavaScript Window Location

- The window.location object can be used to get the current page address (URL) and to redirect the browser to a new page.

- The **window.location.assign()** method loads a new document.

```
<div id="demo"></div>

<script>
document.getElementById("demo").innerHTML =
"Page location is " + window.location.href +
"<br> Page hostname is " + window.location.hostname +
"<br> Page path is " + window.location.pathname;
</script>
```

JavaScript Window History

- The window.history object contains the browsers history.
- Main methods:
  - history.back() - same as clicking back in the browser
  - history.forward() - same as clicking forward in the browser

Finding HTML Elements

- document.getElementById(*id*)
  Find an element by the element id

- document.getElementsByTagName(*name*)
  Find elements by tag name

- document.getElementsByClassName(*name*)
  Find elements by class name

Demo

Try in chrome console:

- document.getElementsByTagName("P").length
- document.getElementsByTagName("P")[0].innerHTML = "Hello World!";
- document.getElementsByTagName("P")[0].style.backgroundColor = "red“
- document.getElementsByClassName(“blue")

Changing HTML Elements

- element.innerHTML = new html content
  Change the inner HTML of an element

- element.attribute = new value
  Change the attribute value of an HTML element

- element.setAttribute(attribute, value)
  Change the attribute value of an HTML element

- element.style.property = new style
  Change the style of an HTML element

Finding HTML Elements by HTML Object Collections

- document.anchors
- document.body
- document.forms
- document.head
- document.images
- document.links
- document.scripts
- document.title

Changing the Value of an Attribute

- To change the value of an HTML attribute, use this syntax:
  - document.getElementById(id).attribute=new value

```
<script>
  document.getElementById("imgId").src ="img.jpg";
</script>
```

Changing HTML Style

- To change the style of an HTML element, use this syntax:
  - document.getElementById(id).style.property=new style

```
<script>
  document.getElementById(“id").style.color =
"blue";
</script>
```

Calling JavaScript Functions

```
function myFunction(p1, p2) {
    return p1 * p2;
   // The function returns the product of p1 and p2
}
```

- Calling a function:

```
document.getElementById("demo").innerHTML = myFunction(4, 3);
```

Accessing a function without () will return the function definition:

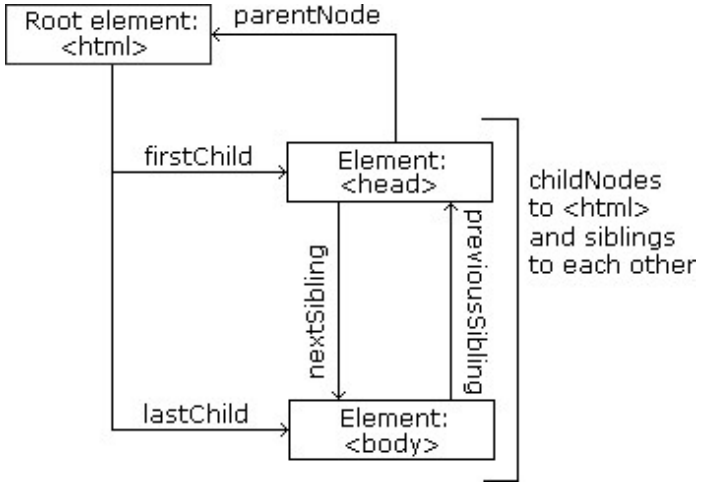
```
document.getElementById("demo").innerHTML = myFunction;
```

JavaScript HTML DOM Navigation

- Everything in an HTML document is a node:
  - The entire document is a document node
  - Every HTML element is an element node
  - The text inside HTML elements are text nodes
  - Every HTML attribute is an attribute node
  - All comments are comment nodes

Node Relationships

- The nodes in the node tree have a hierarchical relationship to each other:
  - In a node tree, the top node is called the root (or root node)
  - Every node has exactly one parent, except the root (which has no parent)
  - A node can have several children
  - Siblings (brothers or sisters) are nodes with the same parent



Navigating Between Nodes

- You can use the following node properties to navigate between nodes with JavaScript:
  - parentNode
  - childNodes[*nodenumber*]
  - firstChild
  - lastChild
  - nextSibling
  - previousSibling

- The nodeName property specifies the name of a node.
- The nodeValue property specifies the value of a node.
- The nodeType property returns the type of node.

Node VS element objects

- The Node object represents a single node in the document tree.
- A node can be an element node, an attribute node, or a text node.

- The Element object represents an element in an HTML document. Elements may contain attributes, other elements

Creating New HTML Elements (Nodes)

```
<div id="div1">
<p id="p1">This is a paragraph.</p>
<p id="p2">This is another paragraph.</p>
</div>

<script>
var para = document.createElement("p");
var node = document.createTextNode("This is
new.");
para.appendChild(node);

var element = document.getElementById("div1");
element.appendChild(para);
</script>
```

Common JS and HTML Events

onchange	An HTML element has been changed
onclick	The user clicks an HTML element
onmouseover	The user moves the mouse over an HTML element
onmouseout	The user moves the mouse away from an HTML element
onkeydown	The user pushes a keyboard key
onload	The browser has finished loading the page

JS and HTML Events

```
<div id="out"></div>
   <script>
     "use strict";

     function displayDate() {
       document.getElementById("out").innerHTML = Date();
       console.log(Date());
     }
   </script>
<button onclick="displayDate();">Show Date</button>
```

Timing Events

- The two key methods to use with JavaScript are:
  - setTimeout(function, milliseconds)
    Executes a function, after waiting a specified number of milliseconds.
  - setInterval(function, milliseconds)
    Same as setTimeout(), but repeats the execution of the function continuously.

setInterval

```
<p id="demo"></p>

<script>
setInterval(myTimer, 1000);

function myTimer() {
  const date = new Date();
  document.getElementById("demo").innerHTML =
date.toLocaleTimeString();
}
</script>
```

More examples on HTML DOM DOCUMENT

- Display all name value pairs of cookies in a document
- Display the domain name of the server that loaded the document Display the date and time the document was last modified Display the title of a document
- Display the full URL of a document Replace the content of a document
- Open a new window, and add some content
- Display the number of elements with a specific tag name

HTML DOM DOCUMENT

index.html

```
4   <p id="demo">Click the button to display the cookies associated with this document.</p>
5   <button onclick="myFunction()">Try it</button>
6
7    <script>
8       function myFunction() {
9           document.getElementById("demo").innerHTML =
10          "Cookies associated with this document: " + document.cookie;
11       }
12  </script>
```

HTML DOM DOCUMENT

index.html

```
4   <button onclick="myFunction()">Try it</button>
5   <p id="demo"></p>
6   <script>
7       function myFunction() {
8        document.getElementById("demo").innerHTML = document.domain;
9       }
10  </script>
```

HTML DOM DOCUMENT

index.html

```
4  <p>This document was last modified <span id="demo"></span>.</p>
5  <script>
6      document.getElementById("demo").innerHTML = document.lastModified;
7  </script>
```

HTML DOM DOCUMENT

index.html

```html
<html>
  <head>
  <title>Learn Confidently</title>
  </head>
<body>

<p id="demo"></p>
<script>
document.getElementById("demo").innerHTML =
"The title of this document is: " + document.title;
</script>

</body>
</html>
```

HTML DOM DOCUMENT

index.html

```
7   <p>The full URL of this document is: <br><span id="demo"></span>.</p>
8   <script>
9      document.getElementById("demo").innerHTML = document.URL
10  </script>
```

HTML DOM DOCUMENT

index.html

```
7   <p id="demo">Click the button to replace this document with new content.</p>
8   <button onclick="myFunction()">Try it</button>
9   <script>
10      function myFunction() {
11        document.open("text/html","replace");
12        document.write("<h2>Learning about the HTML DOM is fun!</h2>");
13        document.close();
14      }
15  </script>
```

HTML DOM DOCUMENT

index.html

```
7   <p>Click the button to open a new window and add some content.</p>
8   <button onclick="myFunction()">Try it</button>
9
10  <script>
11      function myFunction() {
12        var w = window.open();
13        w.document.open();
14        w.document.write("<h2>Hello World!</h2>");
15        w.document.close();
16      }
17  </script>
```

HTML DOM DOCUMENT

index.html

```
7   <p></p>
8   <p></p>
9   <p></p>
10  <p id="demo"></p>
11  <input type="button" onclick="getElements()" value="How many elements tag p?">
12  <script>
13      function getElements() {
14        var x = document.getElementsByTagName("p");
15        document.getElementById("demo").innerHTML = x.length;
16      }
17  </script>
```

HTML DOM FINDING FORM INPUT VALUE

index.html

```
7   <input id="num1"/><br>
8   <input id="num2"/><br>
9   <button onclick="AddTwo()">Add</button>
10  <script>
11      function AddTwo() {
12        var x = document.getElementById("num1").value;
13        var y = document.getElementById("num2").value;
14        var z=parseFloat(x)+parseFloat(y);
15        alert(z);
16      }
17  </script>
```

DOM CREATE ELEMENT APPEND ELEMENT

index.html

```
7   <ol id="MyList"></ol>
8   <input id="Item"/>
9   <button onclick="AppendElement()">Append</button>
10  <script>
11      function AppendElement() {
12        var Item = document.getElementById("Item").value;
13        var MyList = document.getElementById("MyList");
14        let li = document.createElement("li");
15        li.innerHTML=Item;
16        MyList.appendChild(li)
17      }
18  </script>
```

DOM CHANGING ATTRIBUTE VALUE

index.html

```
<img id="image" src="https://cdn.rabbil.com/photos/images/2022/11/04/rabbilVai.png" width="160" height="120">
<button onclick="ChangeSrc()">Change Image Src</button>
<script>
    function ChangeSrc() {
      var image = document.getElementById("image")
      image.src="https://cdn.rabbil.com/photos/images/2022/11/04/whyChoose.png"

    }
</script>
```

DOM QUERY SELECTOR

index.html

```html
<h1>H1</h1>
<h2 id="MyId">H2</h2>
<h3 class="MyClass">H3</h3>
<h4 name="MyName">H4</h4>
<input placeholder="..."/>
<button onclick="Change()">Change</button>

<script>
    function Change() {
      document.querySelector("h1").innerHTML='Hello H1'
      document.querySelector("#MyId").innerHTML='Hello H2'
      document.querySelector('.MyClass').innerHTML='Hello H3'
      document.querySelector('h4[name="MyName"]').innerHTML='Hello H4'
      document.querySelector('input').placeholder='New Placeholder'
    }
</script>
```

Practice

Reading a JSON file and display its content

https://drive.google.com/file/d/1V1zeooK1abQFnB5Im3tL2KDIhbeDsfTQ/view?usp=sharing

Web Application

What are web applications?

- *A web application (web app) is an application program that is stored on a remote server and delivered over the internet through a browser interface.*

- A web browser reads Hypertext Markup Language (HTML) and uses it to render text, images, and controls, such as text input fields, buttons, and hyperlinks.

- The Web browser receives the HTML by connecting to an application server by using the Hypertext Transfer Protocol (HTTP/s).

What are web applications?

- Web applications do not need to be downloaded since they are accessed through a network. Users can access a web application through a web browser, such as Google Chrome, Mozilla Firefox or Safari.

- For a web app to operate, it needs a **web server**, **application server** and **database**.
- Web servers manage the requests that come from a client, while the application server completes the requested task. A database stores any necessary information.

Web server VS Application server

- **A web server‘s** fundamental job is to accept and fulfill requests from clients.
  - It is enough if the client requests are for static content from a website (HTML pages, files, images, video, and so on).
  - A web server’s clients are often Web browsers
- **An application server’s** fundamental job is to provide its clients with access to what is commonly called *business logic*, which generates dynamic content;
  - it’s transforms data to provide the specialized functionality offered by a business, service, or application.
  - An application server’s clients are often applications themselves and can include web servers and other application servers.

Web application structure

A web application is commonly structured as a three-tiered application.

- Frontend or the presentation tier
- *Middle tier or Business logic tier*
- Backend or Data tier

Web-based Application servers

Some of the most well-known Web Application Servers and programming languages.

JavaScript as a client-side language

- JavaScript provides user interactivity with a web page after the page loads, developers commonly use it for some of the following functions:

- Dynamically add, edit, and remove HTML elements and their values
- Validate web forms before submission
- Create cookies to store and retrieve data on a user’s computer for future visits

AJAX: Asynchronous JavaScript And XML

AJAX

- AJAX is not a programming language.
- AJAX just uses a combination of:

  A browser built-in XMLHttpRequest object (to request data from a web server)

  JavaScript and HTML DOM (to display or use the data)

It is used to

- Update a web page without reloading the page
- Request data from a server - after the page has loaded
- Receive data from a server - after the page has loaded
- Send data to a server - in the background

AJAX PACKAGES

The XMLHttpRequest object build in with javascript

Fetch API supported by modern browser's Axios package

JQuery Ajax Superagent

AJAX FETCH API GET REQUEST

index.html

```
8   <button onclick="FetchGetData()">Fetch Get Data</button>
9
10  <script>
11      function FetchGetData() {
12
13      var url="https://crud.teamrabbil.com/api/v1/ReadProduct"
14      var requestOptions = {method: 'GET'};
15
16      fetch(url, requestOptions)
17      .then(response => response.json())
18      .then(result => console.log(result))
19      .catch(error => console.log('error', error));
20
21      }
22  </script>
```

AJAX FETCH API POST REQUEST

index.html

```html
<button onclick="FetchPostData()">Fetch Post Data</button>

<script>
    function FetchPostData() {

    var url="https://crud.teamrabbil.com/api/v1/CreateProduct"

    var data={ Img:"A",ProductCode:"B",ProductName:"C",Qty:"D",TotalPrice:"E",UnitPrice:"F"}
    var requestOptions = {
        method: 'POST',
        headers: {'Accept': 'application/json','Content-Type': 'application/json'},
        body: JSON.stringify(data)
    };

    fetch(url, requestOptions)
    .then(response => response.json())
    .then(result => console.log(result))
    .catch(error => console.log('error', error));
    }
</script>
```

JavaScript Resources

- https://www.codecademy.com/learn/introduction-to-javascript
- https://developer.mozilla.org/en-US/docs/Web/JavaScript
- https://www.freecodecamp.org/learn/javascript-algorithms-and-data-structures/

- https://javascript.info/
- https://www.javascript.com/resources
- http://www.w3schools.com/js/default.asp

Best Free JavaScript Courses

- https://www.codecademy.com/learn/introduction-to-javascript
- https://developer.mozilla.org/en-US/docs/Web/JavaScript
- https://www.freecodecamp.org/learn/javascript-algorithms-and-data-structures/

QUESTIONS